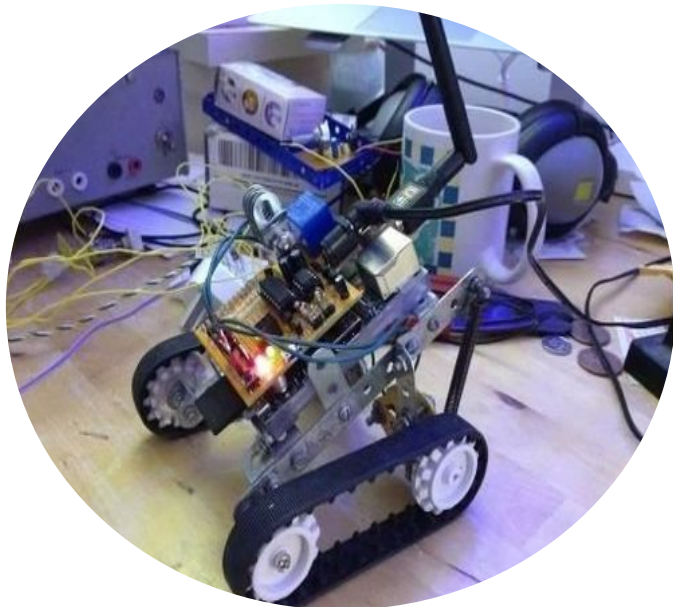
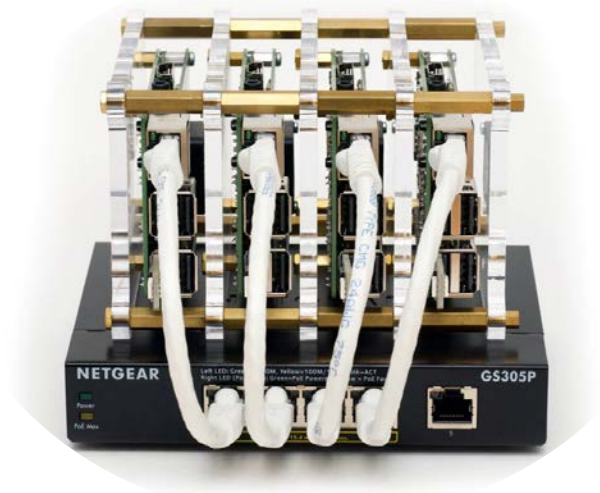




微机原理与微系统实验-树莓派



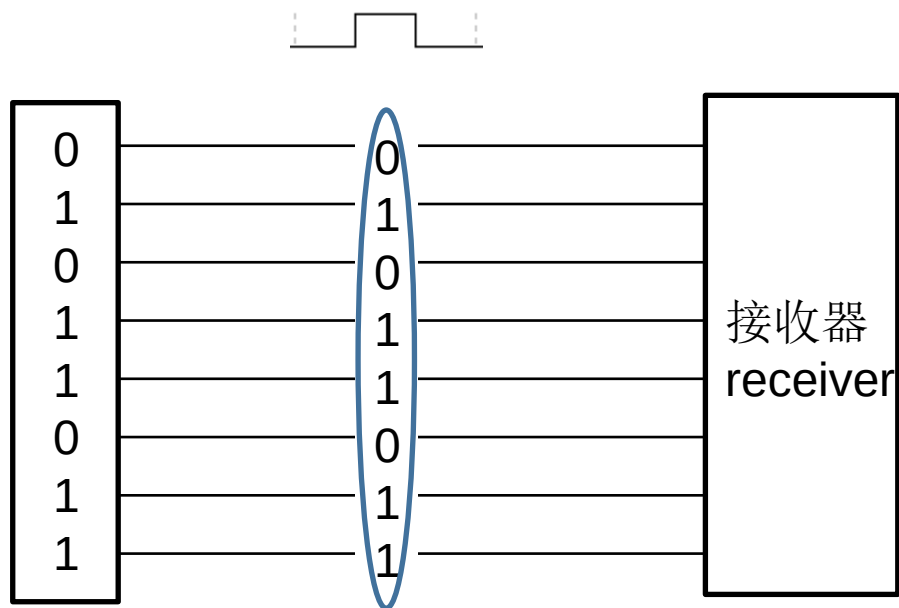
主讲老师：王小静

办公地点：慧园2栋411

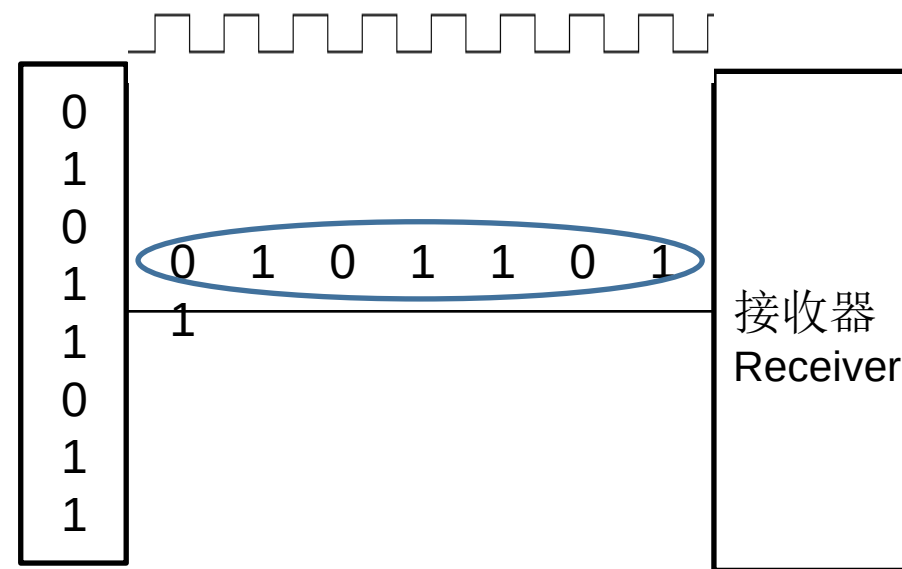
open office hours：周三14:00-16:00

Class 4&5: PCF8591 模数转换器实验

串行、并行总线 (Serial /Parallel Bus)



并行 (Parallel)



串行 (Serial)

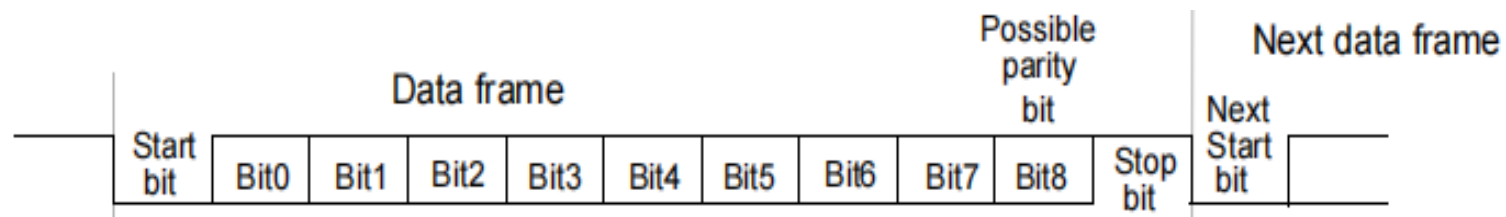
特性	串行通讯	并行通讯
通讯距离	较远	较近
抗干扰能力	较强	较弱
传输速率	较慢	较高
成本	较低	较高

同步串行通信



e.g. SPI I²C

异步串行通信



e.g. UART

同步串行通信和异步串行通信的比较

同步串行通信	异步串行通信
同一个时钟	不同的时钟
一帧以数据块为单位（数据块长度可变）	一帧以字符为单位
效率高，速度快	效率低，速度慢
应用于大批量、高速率数据传输场合	应用于传输速率不高的情况，简单，应用较广

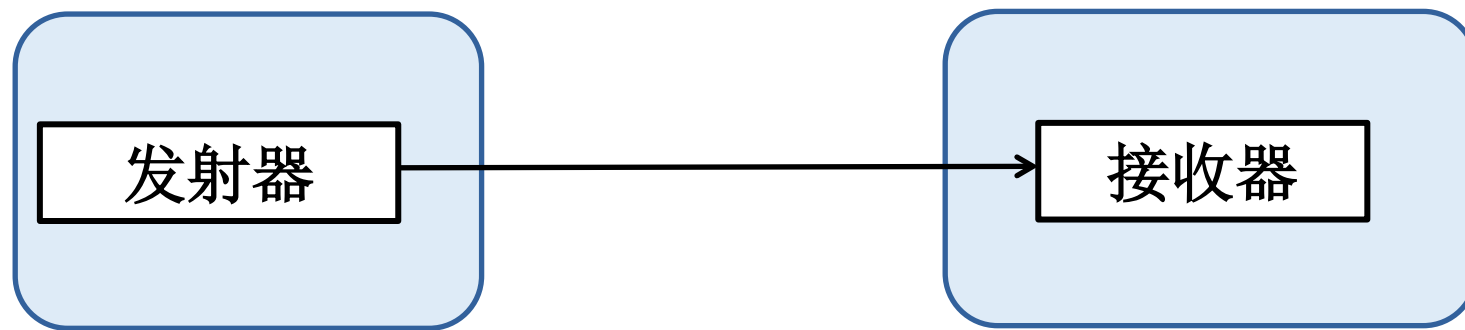
单工/半双工/全双工

单工 (Simplex) : 在单工通信中, 数据只能沿一个固定方向传输, 即从发送器到接收器。类似于广播电台, 听众只能接收而不能发送信息。

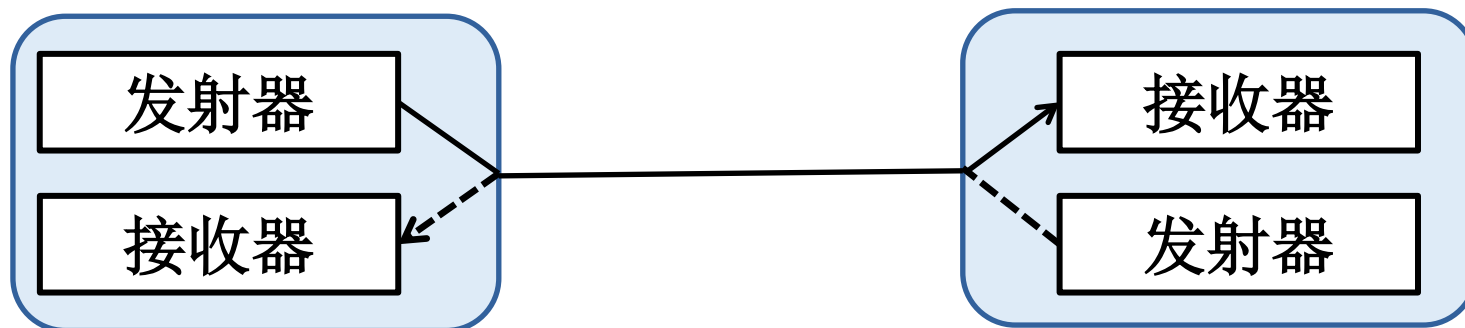
半双工 (Half Duplex) : 半双工通信允许数据在两个方向上传输, 但在同一时刻只能有一个方向的数据传输。也就是说, 设备可以在任何时候发送或接收数据, 但不能同时进行发送和接收。一个常见的半双工应用是无线对讲机。

全双工 (Full Duplex) : 全双工通信允许数据同时在两个方向上传输, 即发送和接收可以同时进行。这种模式下, 通信双方可以不间断地进行数据交换, 提高了通信的效率。全双工通信需要两条独立的通道, 一条用于发送数据, 另一条用于接收数据。以太网和现代电话系统都是全双工通信的例子。

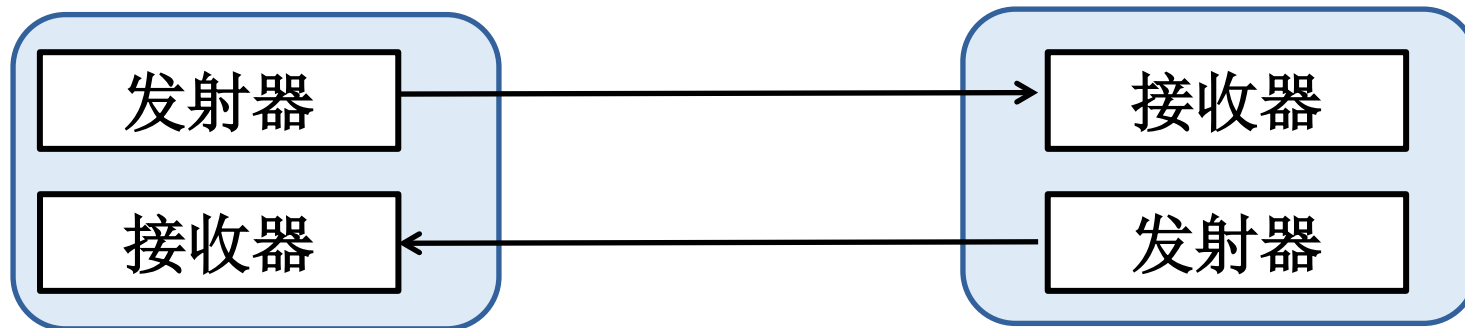
Simplex



Half Duplex



Full Duplex



常用串行通信接口

通信标准	引脚说明	通信方式	通信方向
UART (通用异步收发器)	TXD:发送端 RXD:接受端 GND:公共地	异步通信	全双工
单总线 (1-wire)	DQ:发送/接受端	异步通信	半双工
SPI	SCK:同步时钟 MISO:主机输入，从机输出 MOSI:主机输出，从机输入	同步通信	全双工
I2C	SCL:同步时钟 SDA:数据输入/输出端	同步通信	半双工

I2C串行通信协议 (I2C serial communication protocol)

I2C（Inter-Integrated Circuit）总线是一种广泛使用的串行通信协议，由 Philips 公司（现在的 NXP Semiconductors）在 1980 年代初期开发。I2C 总线允许多个设备通过仅两根线进行通信：串行数据线（**SDA**）和串行时钟线（**SCL**）。这种设计简化了电子设备的内部连线，降低了系统成本，并且提高了可靠性。

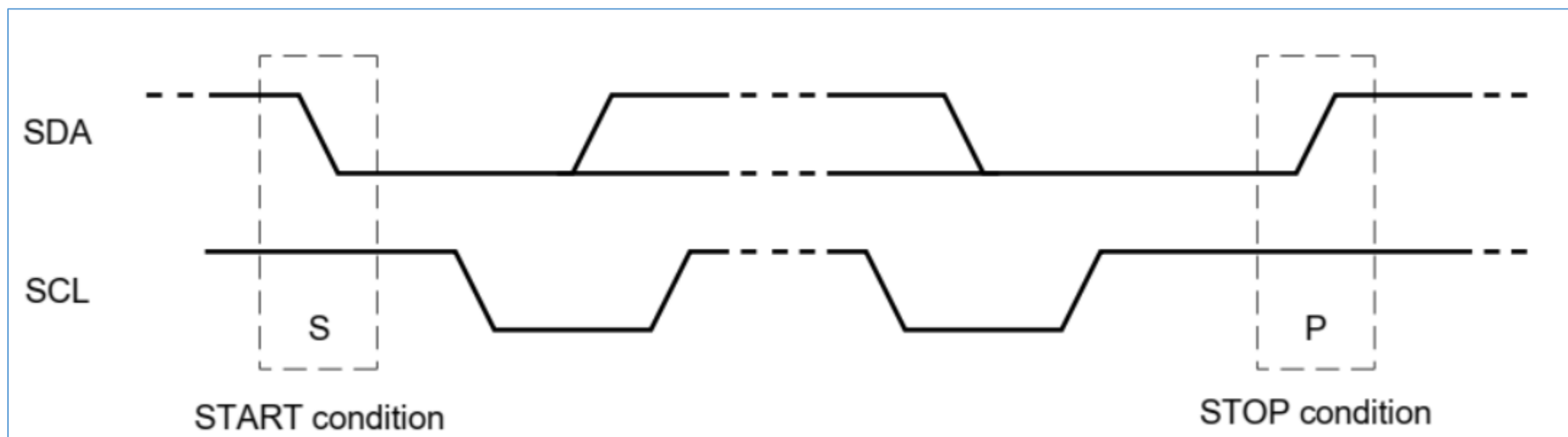
I2C总线的主要特点包括：

- ✓ 两线制总线：仅使用两条线——SDA和SCL，有效减少了连线数量。
- ✓ 多主机支持：支持多个主设备和多个从设备连接到同一总线上。
- ✓ 可变时钟速率：支持多种速率模式，包括标准模式（100kHz）、快速模式（400kHz）、高速模式（3.4MHz）以及超快速模式。
- ✓ 同步通信：数据传输由时钟信号同步控制。
- ✓ 地址分配：每个设备都有一个唯一的地址，可以是7位或10位，从而允许在同一总线上连接多个设备。
- ✓ 应答机制：传输过程中接收设备会发送应答信号，以确认数据包的成功接收。
- ✓ 低功耗设计：适合电池供电设备，具有很低的电流消耗。
- ✓ 易于扩展：可以轻松添加或移除设备，不影响总线上其他设备的通信。

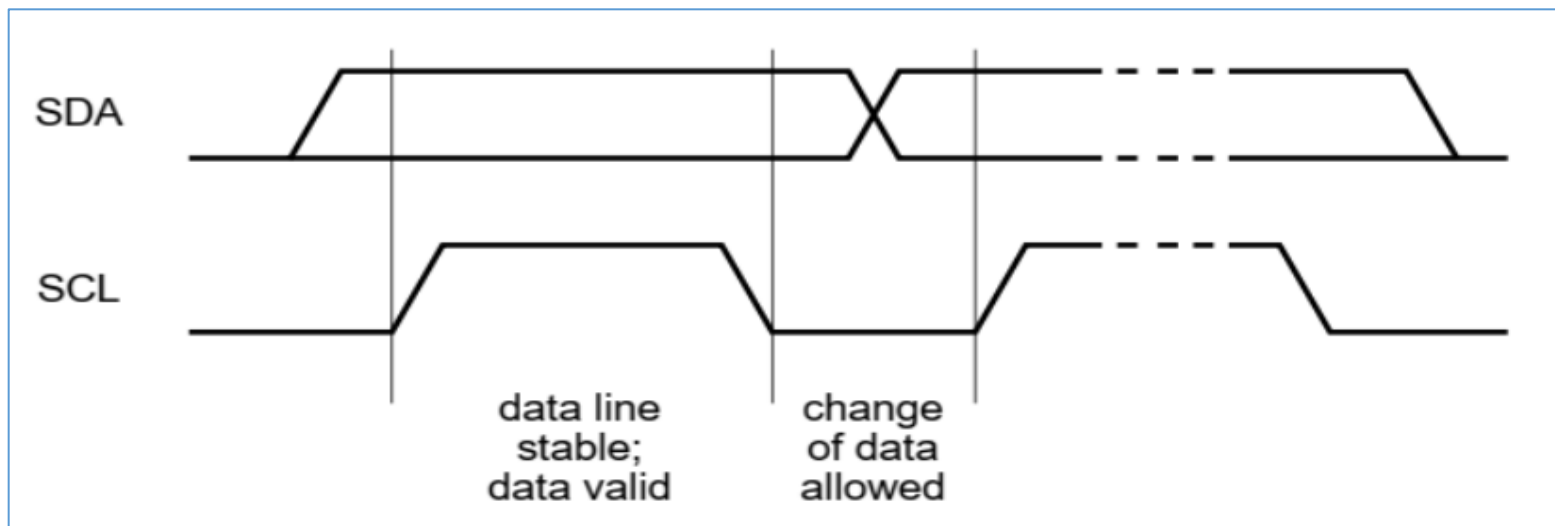
I2C通信过程通常包括以下步骤:

- ✓ 启动条件: 由主设备发起, 通过在SCL为高电平期间, SDA由高到低的跳变来实现。
- ✓ 地址和读写标志: 主设备发送从设备地址 (7/10位), 后跟一个读 / 写位 (1 表示读, 0 表示写)。总线上的所有从设备会将此地址与自身地址比较, 只有匹配的从设备会做出响应。
- ✓ 数据传输: 地址匹配成功之后, 根据读写标志进行数据传输。数据以 8 位字节形式传输。每个字节后跟随一确认位:
 - ◆ ACK: 当接收方成功接收到一个字节的數據后, 它会在第 9 个时钟周期将 SDA 线拉低, 发送一个确认信号 (ACK, Acknowledge)
 - ◆ NACK: 如果接收方无法接收数据 (例如, 从设备忙或地址不匹配), 它会在第 9 个时钟周期保持 SDA 线为高电平, 发送一个非确认信号 (NACK, Not Acknowledge)
- ✓ 停止条件: 由主设备发起, 通过在SCL为高期间SDA由低到高的跳变来实现。

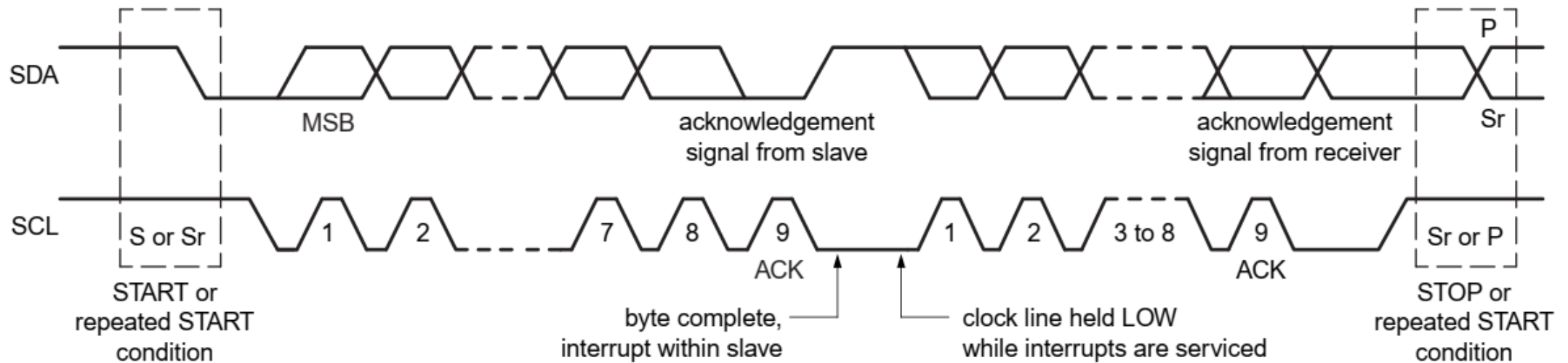
开始与结束：所有事务都是以START(S)开始和STOP(P)结束。当SCL是高时，在SDA线上一个从高到低的转变定义了一个开始条件。当SCL是高时，在SDA线上一个从低到高的转变定义了一个停止条件。



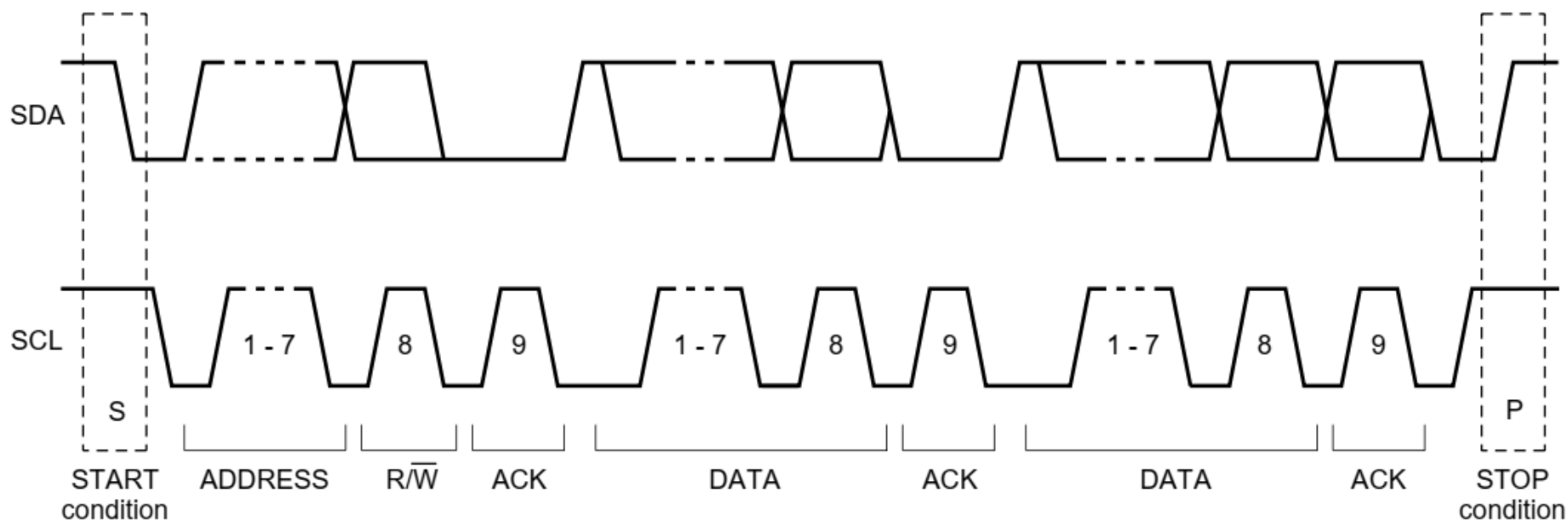
数据有效性： SDA线上的数据在时钟为高时必须稳定。数据线上高或低的状态仅仅在时钟信号SCL为低时改变。每一个数据位的传输都有一个时钟脉冲产生。



字节格式： 放置在SDA线上的每一个字节必须是8bit宽。每一次传输可以被传输的字节数是没有严格限制的。每一个字节后面必须跟一个**ACK bit**位。数据的最高位首先被传输。



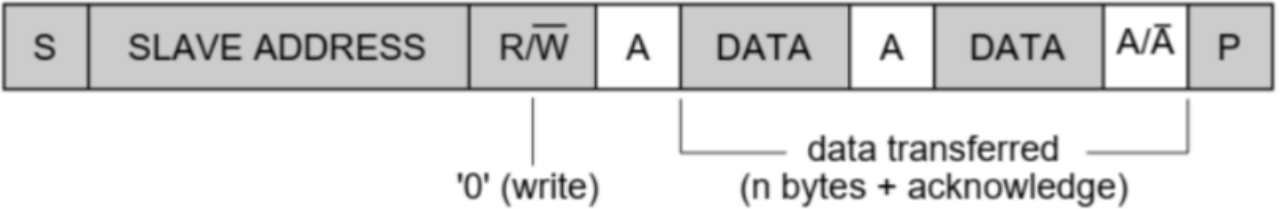
数据传输遵循下图显示的格式：START后，发送slave的地址。地址是7bit，第8bit为数据方向位(R/W)-0表示写传输，1表示读数据。



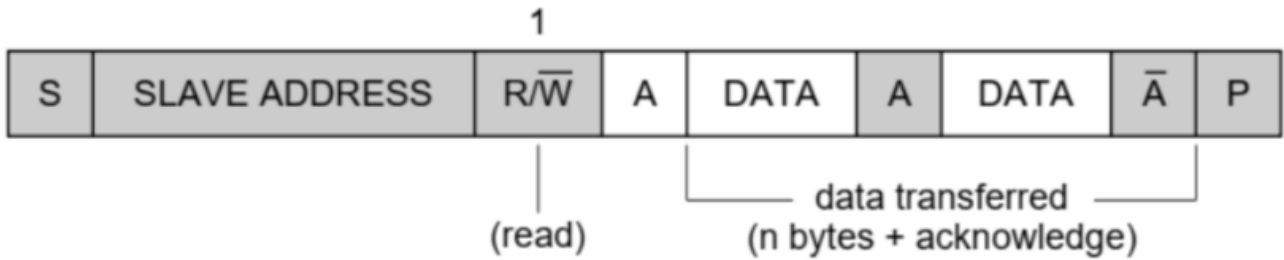
一个完整的字节传输过程

I2C总线的读写数据操作:

写



读



from master to slave

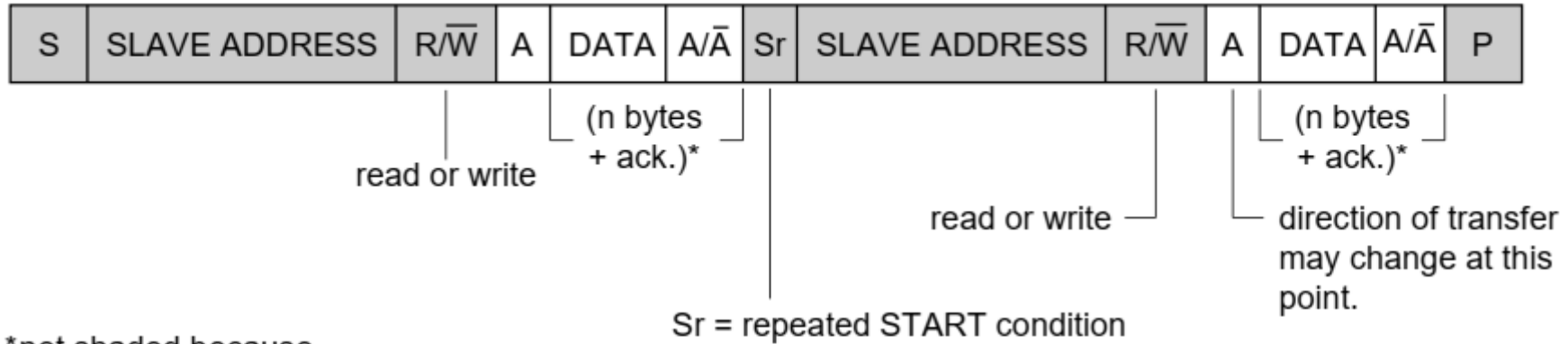
from slave to master

A = acknowledge (SDA LOW)

$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

P = STOP condition



*not shaded because transfer direction of data and acknowledge bits depends on R/ $\bar{W}$ bits.

Combined format



from master to slave



from slave to master

A = acknowledge (SDA LOW)

$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

P = STOP condition

ACK和NACK: 确认发生在每一个字节之后。该确认位允许接收方向发送方发出信号，表示该字节已被成功接收，并且可以发送另一个字节。**Master**生成时钟脉冲，包括告知收到第九个时钟脉冲。

确认信号定义如下：在确认时钟脉冲期间，传送器释放**SDA**线，因此接收器置**SDA**线为低。在时钟脉冲为高期间，接收器保持稳定的低。

在第九个时钟脉冲期间，当**SDA**保持为高，这个被定义为非确认**Not Acknowledge**。然后**master**生成一个**stop**位来退出传输或重新生成一个开始位开始一个新的传输。有五种状况可能导致生成**NACK**位：

- 1.在总线上没有传送器指定地址的接收器，因此没有设备响应一个确认位；
- 2.接收器不能接收或传送，因为它正在执行一些实时的功能，没有准备好与**master**开始通信；
- 3.在传输期间，接收器得到不能理解的数据或命令；
- 4.在传输期间，接收器不能接收更多的数据字节；
- 5.一个**master**接收器必须发送**slave**传送器传输结束的信号。

树莓派smbus库

smbus是Python中用于与I2C（Inter-Integrated Circuit）设备进行通信的库。smbus库提供了一个简单的接口来访问I2C设备，通常用于嵌入式系统，比如Raspberry Pi。

以下是使用smbus库的一些基本步骤：

- ✓ 导入smbus库。
- ✓ 创建一个smbus实例。
- ✓ 使用该实例与I2C设备进行通信。

```
import smbus

# 创建SMBus实例， 1表示I2C的bus number， 在Raspberry Pi中， 通常是1
bus = smbus.SMBus(1)

# 设备地址， 通常是设备的7位地址
DEVICE_ADDRESS = 0x40

# 要写入的数据
data_to_send = 0x01

# 向设备写入一个字节的的数据
bus.write_byte(DEVICE_ADDRESS, data_to_send)

# 从设备读取一个字节的的数据
data_received = bus.read_byte(DEVICE_ADDRESS)

print(f"Received data: {data_received}")
```

在上述代码中，**write_byte**用于向I2C设备发送一个字节的的数据，而**read_byte**用于从I2C设备接收一个字节的的数据。

常用smbus库函数

方法	描述
<code>write_byte(address, value)</code>	向指定的 I2C 设备写入一个字节的數據。
<code>read_byte(address)</code>	从指定的 I2C 设备读取一个字节的數據。
<code>write_byte_data(address, cmd, value)</code>	向设备写入一个字节的數據（先发送一个命令字节）。
<code>read_byte_data(address, cmd)</code>	从设备读取一个字节的數據（先发送一个命令字节）。
<code>write_i2c_block_data(address, cmd, data[])</code>	向设备写入多个字节的數據。
<code>read_i2c_block_data(address, cmd)</code>	从设备读取多个字节的數據。

编码前准备工作

1: 安装smbus库:

你可以使用pip来安装smbus库，在命令行中输入以下命令：

`sudo apt-get update` 更新系统软件源（避免安装失败）

`sudo apt-get install python3-smbus` 安装 **smbus** 库

2: 硬件功能开启

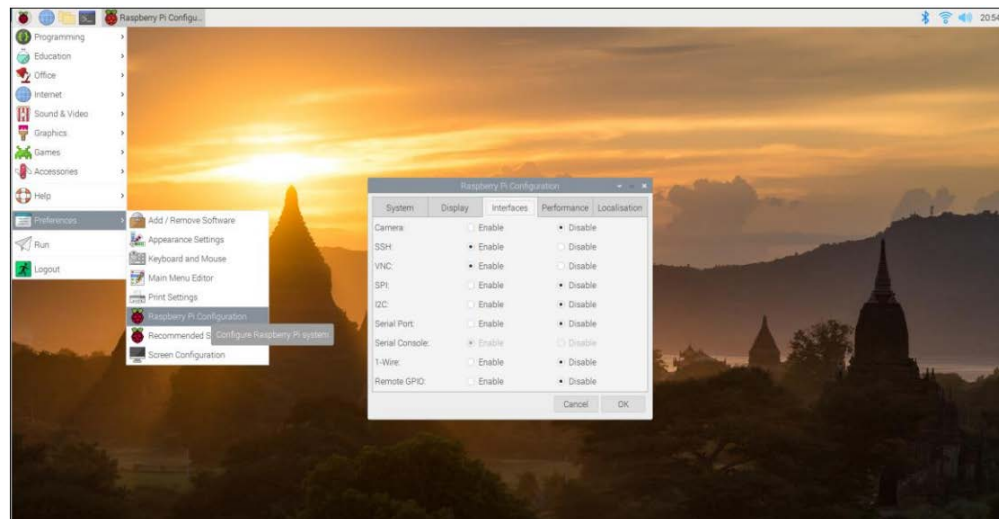
查看i2c是否启动

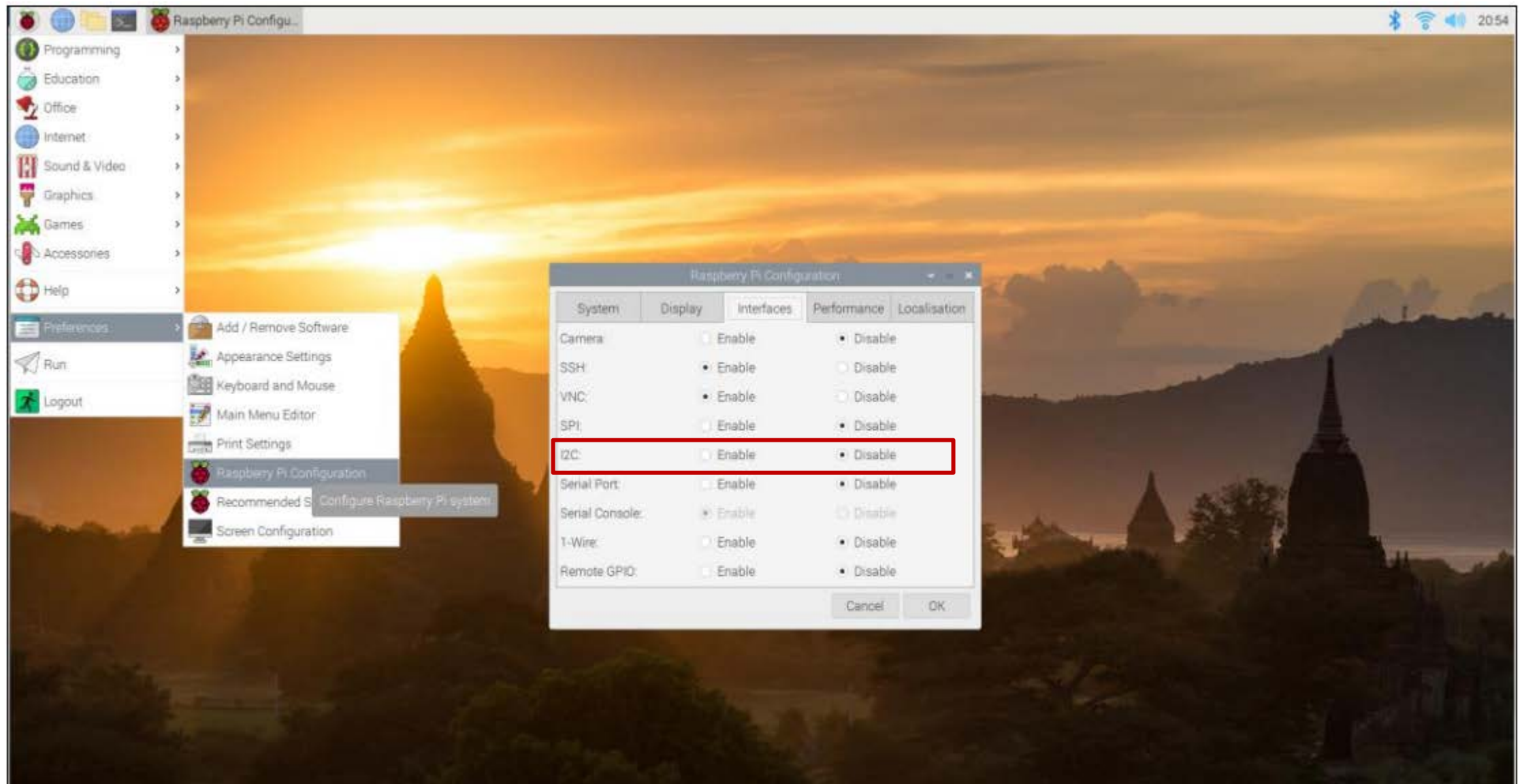
在终端输入:

1: `lsmod` , 可以看到*i2c-bcm2835* 、 *i2c-dev* 等,证明启动成功

2: `ls /dev/i2c-*` ,可以看到 */dev/i2c-1*（有时是 */dev/i2c-0*）或更多，说明 I2C 功能已经启动。

说明：树莓派的硬件接口（比如 I2C、SPI）默认是“禁用”的，目的是节省系统资源。开启后，系统才会加载 I2C 相关的驱动，让 **smbus** 库能找到总线。





编码示例

```
import smbus

bus = smbus.SMBus(1)      # 1 indicates /dev/i2c-1

# write a byte to the specified register
bus.write_byte_data(address, register, value)

# read a byte from the specified register
byte_data = bus.read_byte_data(address, register)

# close the connection
bus.close()
```

在这个示例中，我们首先导入了smbus库，然后创建了一个SMBus实例，指定了使用的I2C bus。然后，我们通过write_byte_data函数向指定的地址和寄存器写入一个字节的数据，并通过read_byte_data函数从指定的地址和寄存器读取一个字节的数据。最后，我们关闭了SMBus连接。

注意：在使用**SMBus**时，你需要知道你要通信的设备的地址和它的寄存器。



Key

Power +	UART	DNC
GND	SPI	
I ² C	GPIO	

	Pin No.	
3.3V	1	2
GPIO2	3	4
GPIO3	5	6
GPIO4	7	8
GND	9	10
GPIO17	11	12
GPIO27	13	14
GPIO22	15	16
3.3V	17	18
GPIO10	19	20
GPIO9	21	22
GPIO11	23	24
GND	25	26
DNC	27	28
GPIO5	29	30
GPIO6	31	32
GPIO13	33	34
GPIO19	35	36
GPIO26	37	38
GND	39	40

5V	1
5V	2
GND	3
GPIO14	4
GPIO15	5
GPIO18	6
GND	7
GPIO23	8
GPIO24	9
GND	10
GPIO25	11
GPIO8	12
GPIO7	13
DNC	14
GND	15
GPIO12	16
GND	17
GPIO16	18
GPIO20	19
GPIO21	20

3.3V PWR	1	2	5V PWR
I2C1 SDA	3	4	5V PWR
I2C1 SCL	5	6	GND
GPIO 4	7	8	TxD
GND	9	10	RxD
GPIO 17	11	12	GPIO 18
GPIO 27	13	14	GND
GPIO 22	15	16	GPIO 23
3.3V PWR	17	18	GPIO 24
SPI0 MOSI	19	20	GND
SPI0 MISO	21	22	GPIO 25
SPI0 SCLK	23	24	SPI0 CS0
GND	25	26	SPI0 CS1
Reserved	27	28	Reserved
GPIO 5	29	30	GND
GPIO 6	31	32	GPIO 12
GPIO 13	33	34	GND
GPIO 19	35	36	GPIO 16
GPIO 26	37	38	GPIO 20
GND	39	40	GPIO 21

“查系统实际启用的 I2C 总线” 来确定参数---- bus = smbus.SMBus(1)

打开树莓派终端，输入命令查看已启用的 I2C 设备文件：

```
ls /dev/i2c-*
```

- 若输出/dev/i2c-1 → 说明系统启用了 I2C1，代码用smbus.SMBus(1),接物理 3/5 针
- 若输出/dev/i2c-0 → 说明启用了 I2C1，代码用smbus.SMBus(0),接物理 27/28 针（需配置启用）
- 若输出/dev/i2c-0 /dev/i2c-1 /dev/i2c-6 → 说明多个总线都启用了，根据接线选对应编号，需配置启用；

PCF8591 模数转换器

PCF8591是一个单片集成、单独供电、低功耗、8-bit CMOS数据获取器件。PCF8591具有4个模拟输入、1个模拟输出和1个串行I²C总线接口。PCF8591的3个地址引脚A0, A1和A2可用于硬件地址编程，允许在同个I²C总线上接入8个PCF8591器件，而无需额外的硬件。在PCF8591器件上输入输出的地址、控制和数据信号都是通过双线双向I²C总线以串行的方式进行传输。

VDD、VSS：电源端。（2.5~6V）

AIN0~AIN3：模拟信号输入端。

A0~A2：引脚地址端

SDA、SCL：I2C 总线的数据线、时钟线。

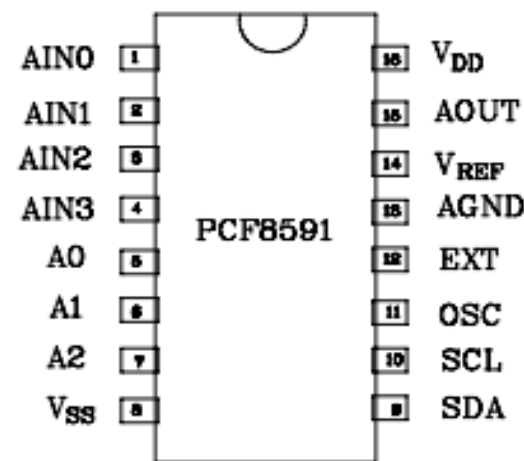
OSC：外部时钟输入端，内部时钟输出端。

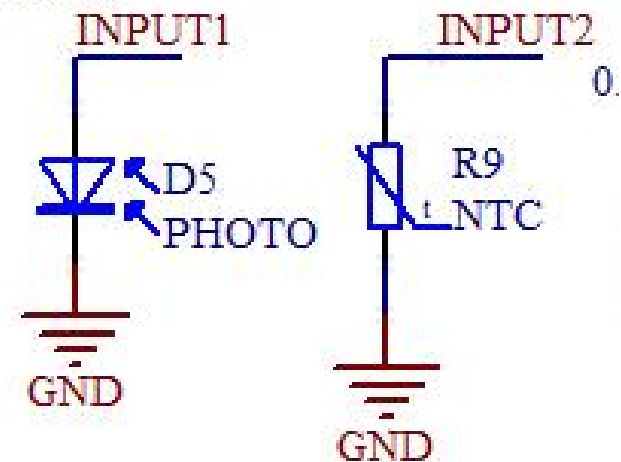
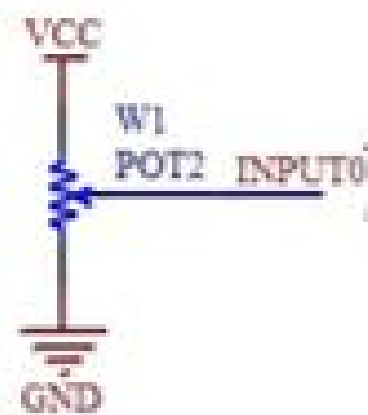
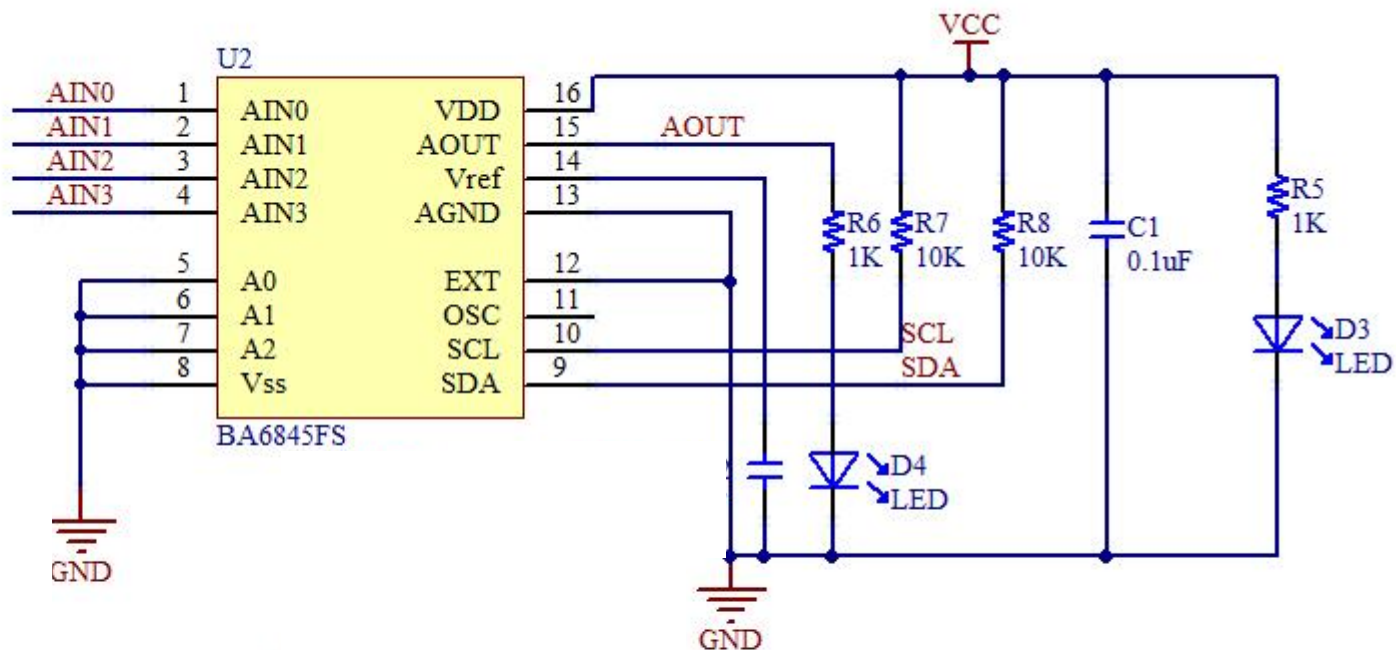
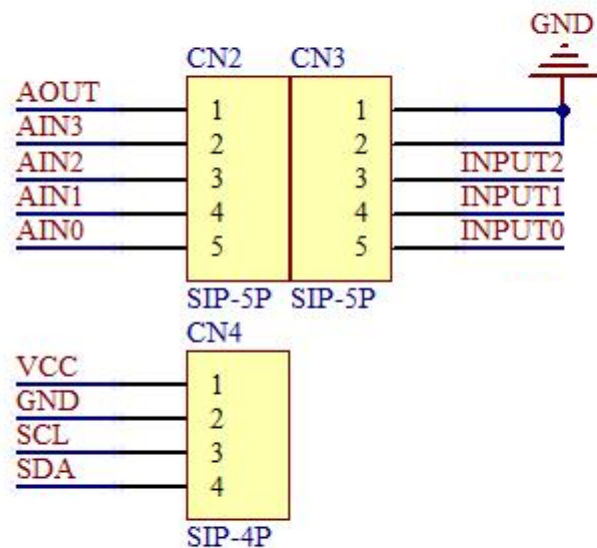
EXT：内部、外部时钟选择线，使用内部时钟时 EXT 接地。

AGND：模拟信号地。

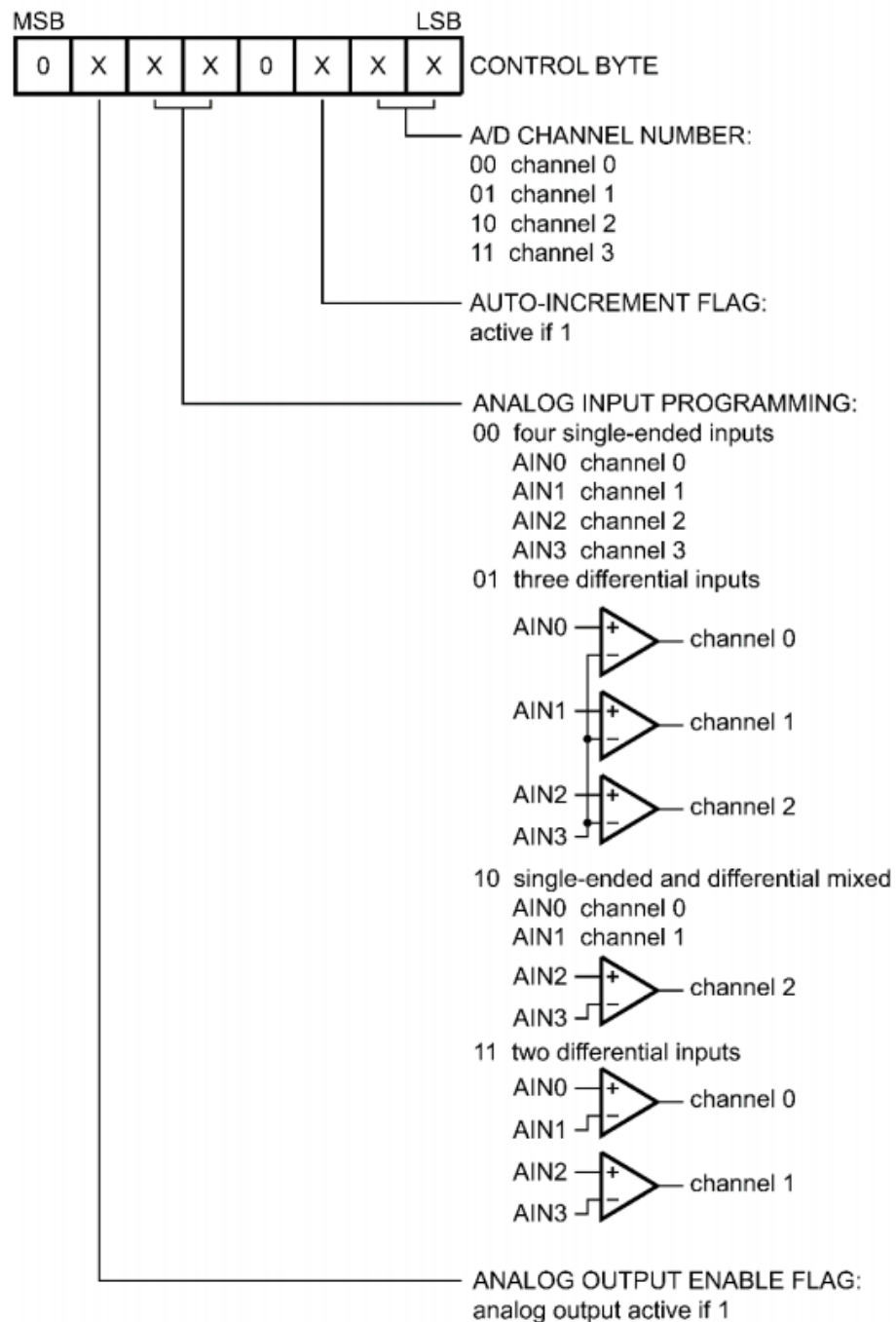
AOUT：D/A 转换输出端。

VREF：基准电源端。



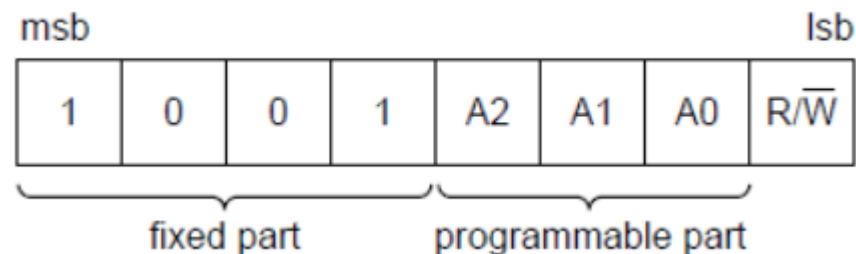


该模块的原理图



备通过向该设备发送有效地址而被激活。地址由固定部分和可编程部分 A0, A1, A2 进行设置。地址必须以邮寄方式发送。
 的第一个字节。地址字节的最后一个位是读/写位，它设置了下列数据

PCF859 设备地址是 0x48（在 7 位地址模式中）或者 0x48（在 8 位地址模式中）



1: 要向PCF8591发送一个数值来控制LED的亮度，

第一个字节为: 0b'10010000即0x90;

第二个字节为: 0b'01000000 即0x40;

第三个字节为: 亮度的赋值

1: 要从PCF8591读取电位器的值，电位器接在AIN0

第一个字节为: 0b'10010001即0x91;

第二个字节为: 0b'00000000 即0x00;

1: 要向PCF8591发送一个数值来控制LED的亮度,

第一个字节为: 0b'10010000即0x90;

第二个字节为: 0b'01000000 即0x40;

第三个字节为: 亮度的赋值

向设备写入一个字节的数据

```
bus.write_byte(DEVICE_ADDRESS, data_to_send)
```

#向设备写入一个字节的数据, 先发送一个命令字节, 然后是数据字节。

```
write_byte_data(address, command, value):
```

本例: `bus.write_byte(0x48,0x40) bus.write_byte(0x48,user_data)`

或者: `bus.write_byte_data(0x48, 0x40, user_data)`

1: 要从PCF8591读取电位器的值, 电位器接在AIN0

第一个字节为: 0b'10010001即0x91;

第二个字节为: 0b'00000000 即0x00;

从设备读取一个字节的数据

```
data_received = bus.read_byte(DEVICE_ADDRESS)
```

本例: `bus.write_byte(0x48,0x00) bus.read_byte(0x48)`

或者: `bus.read_byte_data(0x48, 0x00)`

实验要求

1. 通过笔记本键盘输入一个 led 亮度值，控制 led 亮度（DA 功能）
2. 调节 PCF8591 模块电位器的位置，在程序运行时打印出来当前输入的电压值大小（AD 功能）
3. 实现呼吸灯效果，即借助于 PCF8591 模块让 led 灯在树莓派的控制之下完成由亮到暗、由亮到暗的逐渐变化（DA 功能）
4. 调节 PCF8591 模块电位器改变 LED 灯的亮度（AD-DA 功能）

在Python中，可以使用input()函数来接收用户的输入。如果需要用户输入一个数值，通常需要进行类型转换，因为input()函数默认接收到的数据类型是字符串。

```
# 提示用户输入一个数值
user_input = input("请输入一个数值：")

# 将字符串转换为整数
try:
    number = int(user_input)
    print(f"您输入的数值是：{number}")
except ValueError:
    print("输入的并不是有效的整数，请输入一个整数。")
```

